

05 — Database Documentation

Complete documentation suite (Markdown → PDF with Mermaid diagrams)

Generated: 2026-09-22

Source: repository `docs/` directory

Diagrams rendered with Mermaid 11

Table of Contents

- 05-database/01-database-overview.md
- 05-database/02-schema.md
- 05-database/03-collections-or-tables.md
- 05-database/04-relationships.md
- 05-database/05-indexes.md
- 05-database/06-constraints.md
- 05-database/07-data-lifecycle.md
- 05-database/08-data-dictionary.md
- 05-database/09-erd.md

Database Overview

Technology

MongoDB via Mongoose. No SQL schemas/migrations framework identified.

Connection

Environment variables: MONGO_URI and/or MONGODB_URI (and sometimes DATABASE_URL fallbacks in code).
Optional MONGODB_LOOKUP_FAMILY=4 for IPv4 preference (Atlas).

Deployment model

MongoDB is **external** to Docker Compose (typically Atlas). Containers may reach host Mongo via `host.docker.internal`.

Logical ownership

Domain	Primary service writing schemas
Identity	auth-service models
Orders / logistics / fleet	opms-backend mongoRegistry.js
Products	product-service (mirrors registry)
Parties / rates	party-service (mirrors registry)
Leads / quotations	lead-manager-backend
Work plans	work-planner-backend
Notifications / push	notification-service
Messages	message-service

Inferred: Services share one database/cluster; registries are duplicated. Treat **auth + opms-backend + lead + work-planner + notification + message** as documentation primaries.

Soft-delete

Shared plugin: deletedAt default filter. See [Data Lifecycle](#).

Source

- opms-backend/src/data/mongoRegistry.js
- auth-service/src/models/*
- service-specific registries

Schema Overview

Schemas are defined in Mongoose (JavaScript), not separate migration SQL.

Canonical order domain

`opms-backend/src/data/mongoRegistry.js` — large multi-model file registering Order and related masters.

Identity

`auth-service/src/models/User.js`, `Role.js`, `Department.js`, `Portal.js`, `CompanyInfo.js`

Warning

`opms-backend/docs/OPMS_SCHEMA.md` and `docs/essentials/*` are **partially stale**. Prefer `mongoRegistry` + workflow modules.

Embedded vs referenced

- Order embeds `order_items[]`
- OrderApproval embeds `approval_items[]`
- Cross-entity links use ObjectId refs (Party, Product, User, ...)

Related

- Collections
- Data Dictionary

Collections Catalog

MongoDB collection names generally follow Mongoose pluralization unless `collection:` is set (e.g. `product_kit_items`, `party_product_last_rates`).

Identity (auth-service)

Model	Purpose
User	Accounts, portals, roles
Role	Role catalog
Department	Department catalog
Portal	Portal catalog + default access_roles
CompanyInfo	Branding & legal/tax profile

Catalog / counterparty

Model	Purpose
Party	Customer/supplier
Zone	Territory grouping
Warehouse	Warehouse master
Customer	Legacy/alternate customer (present in registry)
Product	Product / kit
ProductKitItem	Kit composition
ProductGroup / Subgroup / Brand / Manufacturer	Taxonomy
Batch	Inventory batch
PartyProductMapping / Rate / LastRate	Pricing

Order spine

Model	Purpose
Order	Core order
OrderWorkflow	Append-only workflow actions
OrderStatusHistory	Status changes
OrderApproval	Unified approvals
OrderAmendmentUser	Amendment actors
OrderFlag	Flags

Model	Purpose
OrderDispatch	Dispatch
TransportShipment	Shipment
OrderDelivery	Delivery
OrderReturn	Returns
OrderDueSheet	Due sheets
UnbilledOrder	Unbilled tracking
Reminder	Reminders
Attachment	File metadata
ActivityLog	Activity
TransportPlan / TransportPlanOrder	Transport planner
TransportAgent / Vehicle / Driver	Fleet
TermsAndConditions / TermsText	T&C

CRM

Model	Purpose
Lead	Lead
LeadFollowUp	Follow-ups
LeadSource / LeadLostReason	Masters
LeadQuotation	Quotations
FollowUpDigestLog	Digest dedupe

Work planner

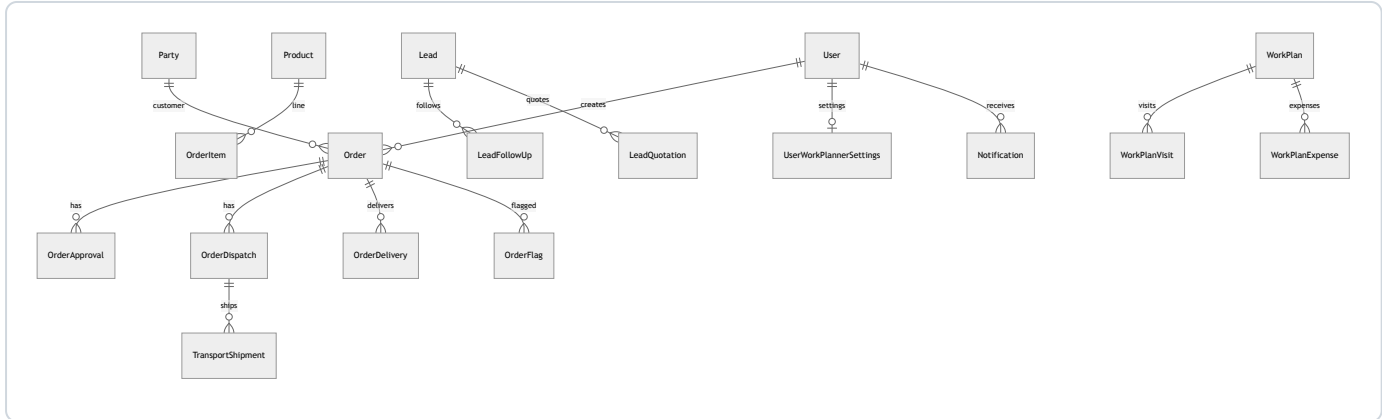
Model	Purpose
WorkPlan	Daily plan
WorkPlanVisit / WorkPlanWork / WorkPlanExpense	Children
UserWorkPlannerSettings	Per-user settings

Comms

Model	Purpose
Notification	In-app notification
PushSubscription	Web push
Message	Outbound message log

Relationships

Logical relationships (Mongo does not enforce FKs):



Notable refs

- Order → party, customer, lead, created_by
- OrderApproval → order
- PartyProductRate → party, product, mapping
- Attachment → entity_type + entity_id (polymorphic)
- TransportShipment → transporter ref **without registered Transporter model** (debt)

Related

- ERD

Indexes

Important indexes observed in schemas (non-exhaustive):

Model	Index notes
User	email unique; company_id
Portal	code unique
Role	code unique
Order	order_no unique; compound indexes including deletedAt
Party	party_name ; sparse gst_no ; featured+active
Product	sku unique sparse; text/ searchable_text
ProductSubgroup	unique (name, group)
Batch	unique (product, batch_no)
PartyProductMapping	unique (party, product)
PartyProductLastRate	unique (party, product)
WorkPlan	unique (sales_user, plan_date) where deletedAt null
WorkPlanVisit	unique (work_plan, sequence)
Lead	lead_no unique
PushSubscription	endpoint unique
Message	indexes on order/recipient/channel/status
FollowUpDigestLog	unique kind+user+digest_date trio
CompanyInfo	is_default

Caveat: OrderDispatch index referencing status while field is dispatch_status may be incorrect — **Requires confirmation.**

Source

Schema `index: true / schema.index(...)` in mongoRegistry and models.

Constraints

Uniqueness

See [Indexes](#) for unique indexes.

Enums (selected)

Order.status

`draft`, `submitted`, `sales_approved`, `finance_review`, `finance_approved`, `finance_rejected`,
`account_review`, `account_approved`, `account_rejected`, `dispatch`, `in_transit`, `delivered`, `closed`,
`cancelled`, `on_hold`

Note: `closed` in schema but absent from transition graph.

Lead.status

`new`, `assigned`, `contacted`, `qualified`, `unqualified`, `follow_up`, `quotation`, `negotiation`, `won`, `lost`,
`converted`

WorkPlan.status

`planned`, `draft`, `submitted`, `approved`, `rejected`, `completed`

Required fields

Vary by model — see Data Dictionary / schema `required: true`.

Application-level constraints

Workflow transition graph; flag blocking; portal RBAC — not DB-enforced.

Data Lifecycle

Soft-delete

- Field: `deletedAt` (Date or null)
- Plugin filters all queries to active docs unless `.withDeleted()`
- Restore clears `deletedAt`

Append-only

- `OrderWorkflow`, `OrderStatusHistory`, `ActivityLog` — no soft-delete in typical config

Message lifecycle

`pending` → `queued` → `sending` → `sent|failed` with attempts metadata

Notification lifecycle

Created unread → marked read (`is_read` , `read_at`)

Due sheets

`active` → `superseded` / `archived`; `is_current` flag

Backup

Not identified automated Mongo backup in repo. Ops must use Atlas/host tooling.

Data Dictionary (selected)

Descriptions only where purpose is clear from field names/code. Unknowns marked.

Collection/Model	Field	Type	Required	Default	Description	Relationship
User	email	String	yes	—	Login identity	unique
User	password	String	yes	—	Hash (select:false)	—
User	portals	Array	no	[]	Portal assignments	→ Portal
User	portals.portal_code	String	—	—	Portal code denorm	—
User	portals.access_roles	[String]	—	—	Roles in portal	—
User	department	String	no	—	Department code/name	—
User	is_active	Boolean	—	—	Active flag	—
Portal	code	String	yes	—	Unique portal code	unique
Portal	access_roles	[String]	—	—	Default roles list	—
Order	order_no	String	—	—	Business number	unique
Order	status	String enum	—	—	Workflow status	—
Order	lifecycle_status	String enum	—	—	Lifecycle summary	—
Order	workflow_stage	String enum	—	—	Stage bucket	—
Order	party	ObjectId	—	—	Counterparty	→ Party
Order	order_items	Array	—	—	Line items	embedded
Order	deletedAt	Date	—	null	Soft-delete	—
OrderApproval	order	ObjectId	yes	—	Parent order	→ Order
OrderApproval	is_admin_approved	Boolean	—	false	Admin signature	—
OrderApproval	is_finance_approved	Boolean	—	false	Finance signature	—
OrderApproval	is_account_approved	Boolean	—	false	Account signature	—
Party	party_type	enum	—	—	customer/supplier/both	—
Party	party_name	String	—	—	Display name	—
Product	product_type	enum	—	—	individual/kit	—
Product	sku	String	—	—	SKU	unique sparse
Lead	status	enum	—	—	Pipeline status	—
Lead	assigned_*	mixed	—	—	Assignment fields	→ User
WorkPlan	plan_date	Date	—	—	Plan day	unique w/ user

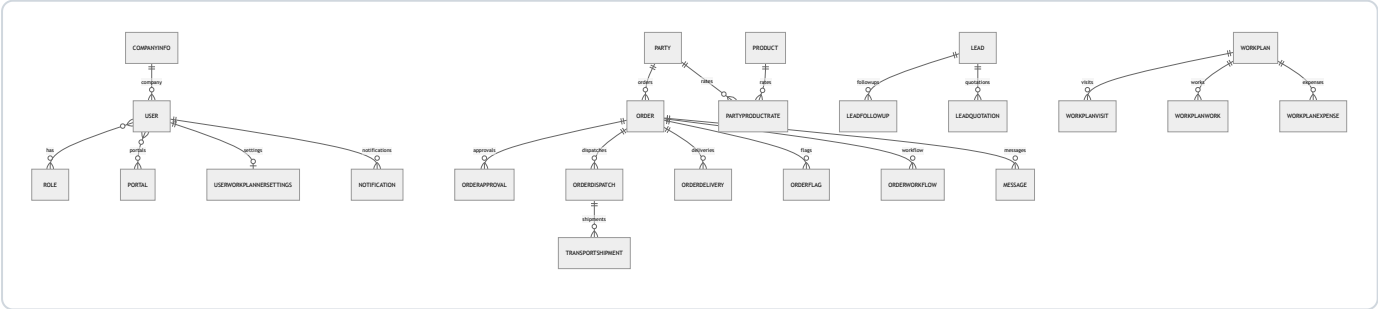
Collection/Model	Field	Type	Required	Default	Description	Relationship
WorkPlan	status	enum	—	—	Plan status	—
Notification	is_read	Boolean	—	—	Read state	—
Message	channel	enum	—	—	email/whatsapp	—
Message	status	enum	—	—	Delivery status	—
Attachment	entity_type	enum	—	—	Owner type	polymorphic
Attachment	entity_id	ObjectId/String	—	—	Owner id	—

Full field lists: see `mongoRegistry.js` and service models — too large to duplicate verbatim; regenerate from source for audits.

Related

- [Collections](#)

ERD



Screenshot: Not available in repository.